To Jepa Or Not To Jepa: That is Le Question

JEPA + SIGReg + Mamba-2 SSM + U-Net Skips + INT4/FP8 QAT + Brotli Compression

A LeWorldModel implementation in combination with Mamba2 SSM and U-Nets to the text field, specifically for this challenge and its bpb evaluation.

Config	Sliding BPB	Standard BPB	Artifact	Compute
BPE best (100k steps, 2.7h)	**1.2064**	1.2235	15.75 MB	8xH100 SXM
BPE best (10min)	1.2566	1.2721	15.50 MB	8xH100 SXM
Byte best (10min)	1.3263	1.3348	15.86 MB	8xH100 SXM

First application of LeWorldModel-style JEPA (Joint Embedding Predictive Architecture) with Mamba2 State Space Modelling to text language. The model learns to predict its own next latent state via MSE while simultaneously training a cross-entropy token prediction head without attention, EMA or stop-gradient, the main benefit of the LeWM paper approach.

Table of Contents

1. Motivation: Why JEPA + SSM?
2. Architecture
3. LeWorldModel Adaptation to Text
4. Training Pipeline
5. Byte vs BPE Tokenization
6. Experimental Results from Ablations
7. Configuration Reference
8. Setup and Run

Motivation: Why JEPA + SSM?

The hypothesis is that self-supervised latent prediction (JEPA) provides a complementary training signal to cross-entropy that improves the encoder's representation geometry, particularly for state-space models where information flow is strictly left-to-right.

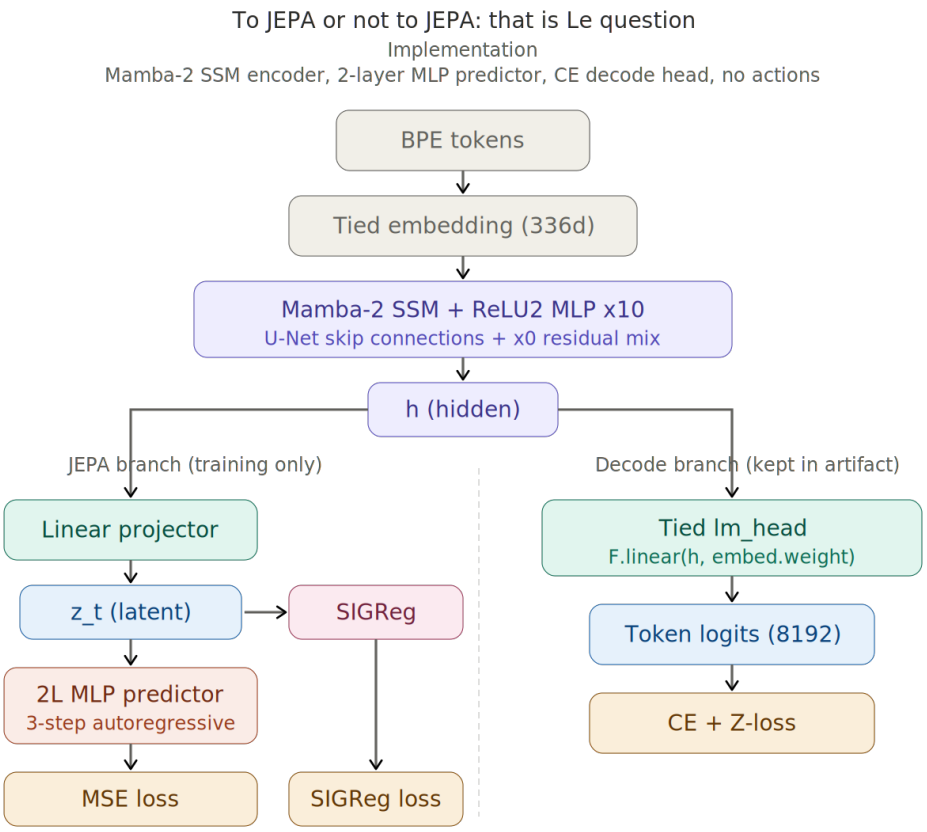
Why JEPA? Standard language model training optimizes a single objective: predict the next token's probability distribution. JEPA adds a second objective: predict what the encoder's internal representation will look like at the next timestep. This forces the encoder to produce representations that are not only useful for token decoding (CE objective) but also smooth and predictable in latent space (JEPA objective). The SIGReg regularizer ensures these representations don't collapse to trivial solutions.

Why Mamba-2? SSMs have a fundamental limitation that makes JEPA particularly interesting: information flows strictly left-to-right. A token at position 100 has no direct access to position 500's representation. The only backward information flow is through gradient propagation during training. JEPA's latent prediction objective explicitly encourages the encoder to produce representations where consecutive positions are related in a structured way - creating an inductive bias toward smooth, predictable state transitions that aligns naturally with how SSMs process sequences.

Why U-Net? Skip connections are arguably more valuable for SSMs than for attention-based models. Attention already provides position-agnostic information mixing, while SSMs don't. U-Net skips give decoder layers access to representations at different processing depths, partially compensating for the lack of bidirectional attention.

The x0 residual mixing provides a gradient highway that bypasses the entire SSM chain.

Architecture



Core Components

Mamba-2 SSM backbone. Each block contains a Mamba-2 selective state space layer (Gu & Dao, 2024) with fused CUDA/Triton kernels for the SSD (Structured State Space Duality) algorithm. The SSM handles sequence mixing - propagating information across positions via a learned recurrent state of dimension d_state=64.

ReLU² MLP. Per-position feed-forward network with squared ReLU activation: `relu(x)²`. This provides the channel mixing that the SSM lacks. The sparsity induced by ReLU² (many exact zeros in the activation) creates a natural fit for quantized weights.

U-Net encoder/decoder with skip connections. The first `num_layers // 2` blocks form the encoder and push their outputs to a LIFO stack. The remaining blocks form the decoder and pop skip connections: `x = x + skip_weight[i] * skips.pop()`. Each block also receives x0 (the post-embedding representation) via a learned residual mix: `x = mix[0] * x + mix[1] * x0`.

Factored tied embedding. Input embedding `[vocab_size, embed_dim]` with learned projections `embed_proj` (embed_dim -> model_dim) and `embed_proj_rev` (model_dim -> embed_dim). The output head reuses the embedding weights via `F.linear(embed_proj_rev(h), token_embed.weight)`. This saves significant parameters for large vocabularies (8192 BPE).

Logit softcap. Polynomial approximation of tanh softcapping (degree 5, cap=15) with Z-loss regularization (`1e-4 * logsumexp²`), keeping logits bounded and gradients sharp through quantization.

JEPA Components (Training-Only - Discarded from Artifact)

Projector: Linear projection `h -> z` mapping hidden states to the JEPA prediction space.

Predictor: 2-layer MLP (`Linear -> GELU -> Linear`) with zero-initialized output. Takes z_t and predicts z_{t+1}. With `JEPA_STEPS=3`, the predictor rolls out autoregressively: predict z_{t+1} from z_t, then z_{t+2} from predicted z_{t+1}, then z_{t+3} from predicted z_{t+2}. Errors compound at each step, which keeps the prediction task challenging and maintains meaningful gradient signal throughout training.

Pred_proj: Linear projection that maps predictor output back to projector space for MSE comparison.

SIGReg: Sketch Isotropic Gaussian Regularizer - enforces that projected embeddings z follow a Gaussian distribution, preventing representation collapse without EMA or stop-gradient. Applied per-timestep across the batch (one whole pass can be done as well, resulting in faster steps, with only a slightly lower result), matching the LeWorldModel paper's specification.

All JEPA components (projector, predictor, pred_proj) are stripped during serialization, contributing zero bytes to the artifact. They exist purely as a training signal that shapes the encoder's representation geometry.

Embedding Tie Modes

Mode	LM Head	Byte (V=256, d=768, e=256)	BPE (V=8192, d=640, e=336)
0	Separate `nn.Linear(dim, V)`	197K	5.24M
1	Tied `F.linear(h, embed.weight)`	0 (shared)	0 (shared)
2	Tied + correction (V x embed_dim)	66K	2.75M
3	Tied + nonlinear adapter (Linear -> GELU -> tied)	197K	215K

Byte mode can afford separate heads (mode 0) since 256 vocab (bytes, also present in Meta's BLT paper) is cheap. BPE mode uses `TIE_EMBEDDINGS=1` (pure tied) to avoid the large 8192-vocab head overhead.

MLP Scheduling (For Reduced Size With Minimal Validation Impact)

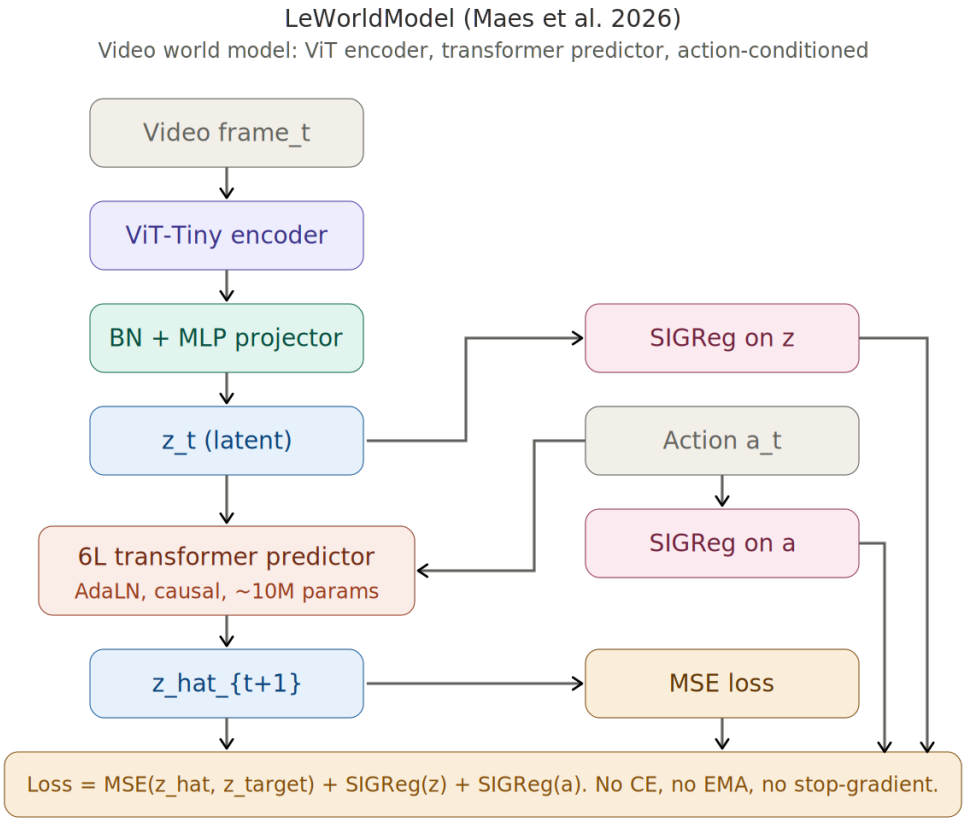
The `MLP_EVERY` parameter controls which blocks receive MLP layers. With `MLP_EVERY=2` on 10 layers:

```
Block 0: SSM + MLP Block 1: SSM only Block 2: SSM + MLP Block 3: SSM only Block 4: SSM + MLP Block 5:
SSM only Block 6: SSM + MLP Block 7: SSM only Block 8: SSM + MLP Block 9: SSM only
```

This halves MLP parameter cost while maintaining per-position nonlinearity every two layers. The modulo pattern ensures no position is ever more than one layer away from an MLP.

It is important to note that the MLP skip technique is possible only at high layer counts, with a minimum of 10, otherwise the model capacity to learn, and, therefore, evaluation metrics collapse. The more layers, the more skips can be introduced and the lower the difference betweem an architecture with skipped MLPs and full MLPs.

LeWorldModel Adaptation to Text



This implementation adapts the LeWorldModel (Maes, Le Lidec, Scieur, LeCun, Balestriero, 2026) from robotics to text. The paper proposes JEPA + SIGReg as a two-term training objective for learning world models from video, replacing contrastive losses and EMA-based methods.

Faithful Adaptations

Core JEPA structure. Encoder produces latent representations z, a predictor forecasts next-step latents via MSE, and SIGReg prevents collapse. The projector/pred_proj structure mirrors the paper's use of projection heads after both encoder and predictor. The `detach_targets=False` default matches the paper's explicit no-stop-gradient stance.

SIGReg implementation. Characteristic function matching via Epps-Pulley quadrature, comparing empirical characteristic function against Gaussian target. Uses fixed random projections (registered as buffers) to avoid CUDA RNG overhead per step.

Deliberate Divergences

Addition of CE loss
 The paper uses exactly two loss terms: MSE prediction + SIGReg. We add cross-entropy token prediction as a third term, because the competition evaluates BPB which requires token-level logits. This makes the JEPA objective an auxiliary regularizer on top of a standard language model, rather than the primary learning signal. The CE loss dominates (circa 99.5% of total loss magnitude by mid-training for BPE), with JEPA providing a mild but consistent shaping signal on the encoder's representation geometry.

Per-timestep SIGReg
 The paper's Algorithm 1 applies SIGReg per-timestep independently across the batch: each position's representations are independently pushed toward Gaussian. Our initial implementation pooled all (B×T) positions together (weaker constraint), for faster steps due to the 10 minute compute budget, which was later changed to per-timestep application with vectorized computation to patch the paper. Integration range updated from [0, 3] to [0.2, 4.0] to match the paper's Appendix A specification.

Simplified predictor
 The paper uses a 6-layer transformer predictor with 16 attention heads, AdaLN conditioning, and 10% dropout (10M params). Our predictor is a 2-layer MLP with zero-initialized output (4M

params). This is appropriate for text where the SSM encoder already captures sequential dependencies, and so the predictor only needs to learn the residual "what changes in the hidden state between adjacent positions."

No action conditioning
 The paper is fundamentally action-conditioned: ■_{t+1} = pred(z_t, a_t). In text there are no actions, so the predictor takes z_t alone and predicts z_{t+1}. This makes the prediction task different - predicting next latent from current without any external conditioning signal.

SIGReg lambda
 The paper uses λ=0.1 and shows robustness for $\lambda \in$ [0.01, 0.2]. Our default is λ=1.0, which is appropriate given different loss scaling from the per-timestep application and different integration ranges. λ at 0.1 or 0.5 proved to be too low to influence training in the expected manner.

Encoder architecture
 Mamba-2 SSM with U-Net skip connections replaces the paper's ViT-Tiny with CLS token pooling. SIGReg is applied to the projector output, not directly to normalized encoder outputs.

Open Question

Whether JEPA is genuinely helping BPB or merely adding noise remains an open empirical question. The JEPA loss drops to ~0.003 by mid-training for BPE, contributing <0.1% of total gradient magnitude. An ablation with `JEPA_WEIGHT=0` would be the most informative experiment to understand its application to text. However, even small representation geometry improvements from JEPA could compound over many training steps, making this difficult to resolve without careful controlled experiments.

Training Pipeline

Quantization-Aware Training (QAT)

All large weight matrices undergo INT4 quantization-aware training from step 1 (`QAT_FRACTION=1.0`). The snap/restore approach is used because Mamba-2's fused CUDA kernels are opaque and straight-through estimators cannot be directly injected into them.

QAT cycle per step:

1. Clone all large matrix weights (full-precision backup)
2. Snap weights to INT4 grid (per-row absmax scaling)
3. Forward + backward with quantized weights (DDP synchronizes gradients)
4. Restore original full-precision weights
5. Apply optimizer update to full-precision weights

This means the model trains against the INT4 quantization grid at every step. A key finding here is that **roundtrip BPB is consistently better than pre-quantization BPB** - the model optimizes specifically for the quantized grid, and the full-precision weights contain noise that quantization clips away.

FP8 QAT for medium-sized matrices (embeddings, non-SSM 2D params): simulated QAT via `param.data.copy_(param.data.to(float8_e4m3fn).to(param.dtype))` straight-through estimator.

Everything else is stored as **BF16** to keep precision while reducing size to fit the compressed 16MB size budget.

Serialization

INT4 packing: Per-row absmax scaling -> signed INT4 values -> np.packbits for bit-level packing. Scales stored as BF16.

Multi-compressor selection: Each artifact is compressed with LZMA (preset=9), Zstandard, and Brotli. The smallest is automatically selected. Brotli consistently wins for any INT4/5/6+FP8/BF16 mixed artifacts.

Training-only param stripping: Projector, predictor, and pred_proj are excluded from the artifact. Only eval-path parameters are serialized.

Optimizers

Muon (matrix params): Newton-Schulz orthogonalization with 3 backend steps, momentum 0.95, warmup from 0.85 over 500 steps.

AdamW (scalar params + embeddings): Fused implementation, β■=0.9, β■=0.95, separate learning rates for scalars (0.01) and embeddings (0.01).

Ghost Warmup

10-step warmup that runs forward+backward to prime optimizer momentum buffers, then restores model weights (but keeps momentum). Adam step counters are reset to zero so bias correction starts fresh. This gives the optimizer "warm" second-moment estimates from step 1 without biasing the model toward warmup data.

Temperature Scaling

Two-phase search calibrated on **training data** (not validation - avoiding data leakage):

1. Coarse grid: [0.5, 0.7, 0.85, 0.95, 1.0, 1.1, 1.3, 1.5, 2.0]
2. Fine grid: ±0.06 around best in steps of 0.02

It works similarly to a Random Search Grid with a Localised Grid Search applied on top. Optimal temperature is typically 1.00-1.02 for this architecture.

Byte vs BPE Tokenization

The codebase supports dual-mode operation via `TOKENIZER=byte` (256 vocab) or `TOKENIZER=bpe` (8192 vocab), switching data loading, model vocabulary, and BPB calculation with a single flag.

Why BPE Dominates on BPB

The BPB formula difference is fundamental. BPE tokens cover ~4.2 bytes on average, so:

- **Byte:** `BPB = loss_nats / ln(2)` - each prediction is one byte
- **BPE:** `BPB = (loss_nats / ln(2)) × (tokens / bytes)` - the tokens/bytes ratio (~0.24) dramatically reduces BPB

To match the ternary's 1.157 BPB with bytes, you'd need val_loss ~0.58, which proves extraordinarily difficult.

How BPE Affects JEPA

With bytes, predicting z_{t+1} from z_t is often trivially easy - character bigram/trigram patterns embedded in the SSM's hidden state. JEPA cos_sim hits 0.999 quickly, and the loss drops to ~0.003 by mid-training. The prediction task is mechanically solved due to the complexity of the purpose built architecture.

With BPE, predicting z_{t+1} is genuinely hard. Each token encodes ~4 bytes of text, and the next token could be one of thousands. The JEPA loss stays meaningful (~0.004-0.005) through most of training, providing a stronger regularization signal. Multi-step prediction (`JEPA_STEPS=3`) is more natural with BPE because 3 tokens ahead is ~12 bytes of text - a meaningful prediction horizon.

BPE Architecture Adaptations

- Sequence length reduced from 4096 (byte) to 1024 (BPE) - same text coverage, 4x fewer positions, significant speed improvement
- Tied embeddings essential for BPE (8192×256 = 2M shared params vs 8192×640 = 5.2M for separate head)

- Token-to-byte lookup tables (from previous ternary implementation #PR640) for accurate BPB calculation
- Predictor hidden mult increased from 2 to 4 for harder BPE prediction task (training-only, zero artifact cost)

Experimental Results From Ablations

BPE Mode - 10-Minute Runs (8×H100 SXM, 599s)

Config	Layers	Dim	MLP	MLP Every	Expand	Tie	BPB (std)	BPB (RT)	Artifact	Steps	ms/step
Best 10min	10	640	4	2	1	1	1.3080	**1.2721**	15.50 MB	6,090	98
12L every=2	12	640	3	2	1	1	1.2752	1.2752	15.35 MB	5,500	109
10L uniform	10	576	3	1	1	1	1.3224	1.2702	15.42 MB	5,820	100
8L uniform	8	640	3	1	1	1	1.3440	1.2715	15.26 MB	6,470	90
10L every=2	10	640	3	2	1	1	1.3221	1.2854	13.18 MB	6,370	94
8L every=2	8	640	3	2	1	1	1.3492	1.3051	10.99 MB	7,380	80
First BPE	10	512	2	1	2	old	1.3439	1.2861	15.50 MB	6,450	90
tie=3 adapter	8	640	3	1	3	l=2	1.3684	1.2850	15.19 MB	6,600	90
10L full	10	640	3	1	1	1	1.2964	1.2565	18.47 MB	5,580	105
13L every=2	13	640	3	2	1	1	1.3074	1.2652	16.93 MB	5,100	117

BPE Mode - Extended Run (100k steps, ~2.7 hours)

Metric	Value
Config	640d, 10L, mlp=4, mlp_every=2, expand=1, embed=336, tie=1
val_bpb (standard)	1.2235
val_bpb (sliding, stride=16)	**1.2064**
val_bpb (roundtrip)	1.2235
Optimal temperature	1.02
Artifact + code	15.75 MB / 16.00 MB
Steps completed	100,000
ms/step	97.4
Total training time	2.71 hours
Eval params	32,816,684

Metric	Value
Discarded (JEPA) params	4,099,200
Compression	INT4 + FP8 + Brotli

Byte Mode - 10-Minute Run (8×H100 SXM, 600s)

Metric	Value
Config	768d, 10L, mlp=3, mlp_every=2, expand=1, embed=256, tie=1
val_bpb (standard)	1.3348
val_bpb (sliding, stride=128)	**1.3263**
val_bpb (roundtrip)	1.3348
Optimal temperature	1.00
Artifact + code	15.86 MB / 16.00 MB
Steps completed	5,730
ms/step	104.9
Eval params	37,007,208
Discarded (JEPA) params	5,902,080
Sequence length	8,192
Compression	INT4 + FP8 + Brotli

Byte Mode - INT4 Lambda Sweep (10min, 512d 10L expand=2)

Lambda	ce_weight	BPB (RT)	Artifact	Notes
1.0	1.0	**1.3276**	12.48MB	Best byte-level
5.0	1.0	1.3339	12.40MB	Over-regularized
5.0	0.5	1.3477	12.32MB	CE downweighted hurts BPB

Higher SIGReg lambda forces the encoder to prioritize Gaussian structure over CE decodability, costing BPB. Lambda=1.0 provides sufficient anti-collapse pressure for byte-level data.

Key Findings: JEPA Steps

JEPA Steps	BPB (RT)	Notes
1	1.3315	Single-step prediction saturates quickly
3	**1.3276**	Multi-step rollout keeps gradient signal alive

Multi-step prediction (JEPA_STEPS=3) maintains meaningful JEPA loss through error compounding, providing a richer training signal at the cost of ~3x predictor compute per step (~5-10% slowdown).

Key Finding: MLP Every

Config (8L, 640d, mlp=3)	MLP Every	BPB (RT)	Artifact	Steps
All blocks have MLP	1	**1.2715**	15.26MB	6,470
Alternate blocks	2	1.3051	10.99MB	7,380

MLP on every block outperforms alternating despite fewer training steps. Per-position nonlinearity is more valuable than the extra steps gained from smaller model. However, at 10 layers with `MLP_EVERY=2`, the quality gap narrows significantly as more SSM depth compensates.

Key Finding: Expand Factor

Config	Expand	BPB	Notes
640d 8L mlp=3	1	1.2715	15.26MB, wider MLP compensates
512d 10L mlp=2	2	1.2861	15.50MB, larger Mamba2 in_proj

Expand=1 with wider MLP (mlp=3) outperforms expand=2 with narrower MLP (mlp=2). The MLP's per-position channel mixing is more valuable per parameter than Mamba2's expanded internal dimension.

Key Finding: Roundtrip Improvement from QAT

Across all runs, the quantized (roundtrip) BPB is **better** than the full-precision pre-quantization BPB by 0.01-0.06. This occurs because INT4 QAT from step 1 means the model optimizes for the quantized weight grid. The full-precision weights contain noise that the quantization grid clips away - the quantized weights are what the model actually learned to use.

Key Finding: Sliding Window Eval

Config	Standard BPB	Sliding BPB	Delta
BPE 10min best	1.2721	1.2566	–0.016
BPE 100k steps	1.2235	1.2064	–0.017
Byte 10min best	1.3348	1.3263	–0.009

Sliding window evaluation (stride=16 for BPE, stride=128 for byte) provides consistent improvement by allowing the SSM to build up its recurrent state before scoring, reducing cold-start penalty at sequence boundaries. The improvement is larger for BPE because seq_len=1024 creates more boundary artifacts than seq_len=8192 for bytes.

10-Minute vs Extended Training (BPE)

Metric	10min (6,090 steps)	100k steps (2.7h)	Improvement
Standard BPB	1.2721	1.2235	–0.049
Sliding BPB	1.2566	1.2064	–0.050
val_loss	3.3782	3.1598	–0.219

Extended training provides substantial gains. The model has not converged at 100k steps - the loss curve is still declining, suggesting further improvement with more compute. The 10-minute constraint is the primary bottleneck for this architecture.

Configuration Reference

Best BPE Config (10 minutes)

```
MODEL_DIM=640 NUM_LAYERS=10 D_STATE=64 EXPAND=1 MLP_MULT=4 MLP_EVERY=2 EMBED_DIM=336 VOCAB_SIZE=8192
TIE_EMBEDDINGS=1 TRAIN_SEQ_LEN=1024 TOKENIZER=bpe JEPA_WEIGHT=1.0 JEPA_STEPS=3 CE_WEIGHT=1.0
SIGREG_LAMBDA=1.0 QUANT_BITS=4 FP_STORAGE=FP8 LOGIT_SOFTCAP=15 SOFTCAP_TYPE=poly MATRIX_LR=0.02
SCALAR_LR=0.01 EMBED_LR=0.01 MUON_BACKEND_STEPS=3 WARMDOWN_FRACTION=0.15
```

Best Byte Config (10 minutes)

```
MODEL_DIM=768 NUM_LAYERS=10 D_STATE=64 EXPAND=1 MLP_MULT=3 MLP_EVERY=2 EMBED_DIM=256 VOCAB_SIZE=256
TIE_EMBEDDINGS=1 TRAIN_SEQ_LEN=8192 TOKENIZER=byte JEPA_WEIGHT=1.0 JEPA_STEPS=3 CE_WEIGHT=1.0
SIGREG_LAMBDA=1.0 QUANT_BITS=4 FP_STORAGE=FP8 LOGIT_SOFTCAP=15 SOFTCAP_TYPE=poly MATRIX_LR=0.02
SCALAR_LR=0.01 EMBED_LR=0.01 MUON_BACKEND_STEPS=3 WARMDOWN_FRACTION=0.15
```

Parameter Budget Analysis (BPE Best Config)

Component	Per Block	10 Blocks	Quantization	Compressed
Mamba-2 in_proj	760K	7.6M	INT4	~3.8MB
Mamba-2 out_proj	410K	4.1M	INT4	~2.1MB
MLP fc (5 blocks)	1.64M	8.2M	INT4	~4.1MB
MLP down (5 blocks)	1.64M	8.2M	INT4	~4.1MB
Token embed (8192×336)	-	2.75M	FP8	~2.8MB
Embed proj/rev	-	0.43M	INT4	~0.2MB
Skip weights, scalars	-	~40K	BF16	~0.08MB
Total eval		**32.8M**		**~15.7MB**
Discarded (JEPA)		4.1M	-	0

Setup and Run

```
# Full environment setup (conda, dependencies, dataset, tokenizer) bash setup_jepa.sh # BPE mode (10
minutes, 8xH100) bash run_jepa_bpe.sh # Byte mode (10 minutes, 8xH100) bash run_jepa_ssm.sh #
Extended BPE run (unconstrained compute) MAX_WALLCLOCK_SECONDS=0 ITERATIONS=100000 bash
run_jepa_bpe.sh
```

Acknowledgements

Architecture builds on Mamba-2 (Gu & Dao, 2024), LeWorldModel JEPA (Maes et al., 2026), and SIGReg (same). The U-Net skip connections, Muon optimizer, polynomial softcap, and FP8 QAT are adapted from our ternary transformer submission. The BPE tokenizer and token-to-byte lookup tables are shared with the ternary codebase.